
Q1. Define Data Structure and explain with an example.

Answer:

1. Formal Definition

A **Data Structure** is a specialized format for organizing, processing, retrieving, and storing data in a computer so that operations can be performed efficiently. It is the mathematical or logical model of a particular organization of data.

2. Classification of Data Structures

Data structures are generally categorized into two main types:

- **Linear Data Structures:** Elements are arranged in a sequential order where each element is connected to its previous and next adjacent elements (e.g., Arrays, Stacks, Queues, Linked Lists).
- **Non-Linear Data Structures:** Data is organized in a hierarchical or interconnected way, allowing for complex relationships (e.g., Trees, Graphs).

3. Practical Example: A Library System

Consider a **Stack** data structure.

- **Concept:** A stack follows the **LIFO (Last-In, First-Out)** principle.
- **Real-world Analogy:** Imagine a stack of books on a desk. You can only place a new book on the top (**Push**) and take the topmost book off (**Pop**).
- **Efficiency:** This structure is ideal for tasks like "Undo" operations in word processors, where the most recent action is the first one reversed.

Q2. Define Algorithm and explain the concept with an example.

Answer:

1. Formal Definition

An **Algorithm** is a finite, step-by-step procedure or set of well-defined instructions to solve a specific problem or perform a calculation. It serves as a blueprint for writing a computer program.

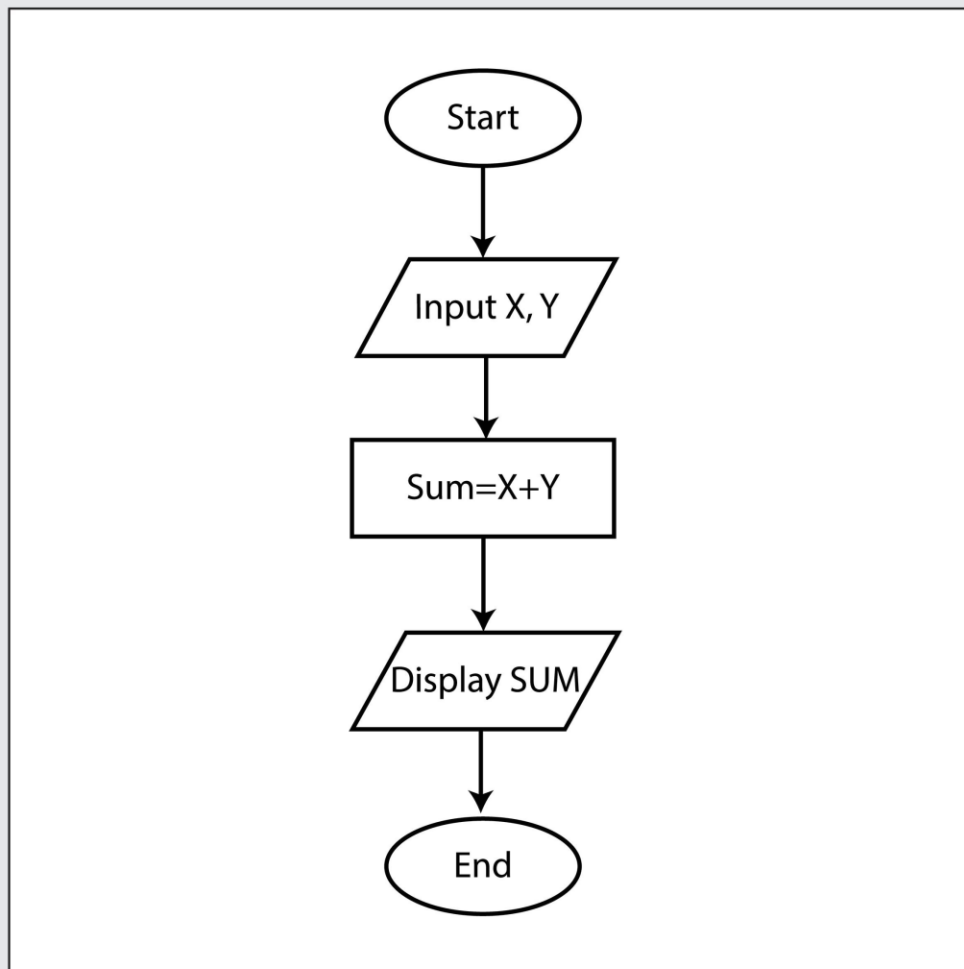
2. Essential Properties of an Algorithm

For an algorithm to be effective in a university context, it must possess:

- **Input/Output:** It must take zero or more inputs and produce at least one output.
- **Definiteness:** Every step must be clear and unambiguous.
- **Finiteness:** It must terminate after a specific number of steps.
- **Effectiveness:** Every instruction must be basic enough to be executed by a human or machine.

3. Example: Calculating the Average of Three Numbers

1. **Start.**
2. **Input:** Read three numbers A , B , and C .
3. **Process:** Calculate the sum $S = A + B + C$.
4. **Process:** Calculate Average $Avg = S / 3$.
5. **Output:** Display Avg .
6. **Stop.**



Shutterstock

Q3. Explain Linear Search in an array with an example.

Answer:

1. Definition

Linear Search is the most basic searching technique where you scan every element of a list sequentially until the desired element is found or the list ends.

2. Algorithm Steps

1. Start from the first element (index 0).
2. Compare the current element with the **Target Value**.
3. If a match is found, return the index of that element.
4. If no match is found, move to the next index and repeat.
5. If the end of the array is reached without a match, the search is unsuccessful.

3. Numerical Example

Search for '7' in Array: [10, 5, 8, 7, 2]

- **Step 1:** Is $10 == 7$? No.
 - **Step 2:** Is $5 == 7$? No.
 - **Step 3:** Is $8 == 7$? No.
 - **Step 4:** Is $7 == 7$? **Yes!** Return Index 3.
-

Q4. Define an Array and explain how to calculate the address in a 2D array using Row-Major Order.

Answer:

1. Definition

An **Array** is a collection of similar data elements stored in contiguous memory locations. A **2D Array** is visualized as a table with rows and columns, but it is stored linearly in the computer's memory.

2. Row-Major Order (RMO) Concept

In RMO, the elements are stored in memory **row by row**. This means all elements of the first row are stored first, followed by all elements of the second row, etc.

3. Address Calculation Formula

To find the memory address of an element at index $A[i][j]$:

$$\text{Address}(A[i][j]) = \text{BaseAddress} + W \times [n \times (i - L_r) + (j - L_c)]$$

- **BaseAddress:** The address of the very first element $A[0][0]$.

- **W:** Size of each element in bytes (e.g., 2 or 4).
 - **n:** Total number of columns in the array.
 - **i, j:** Current row and column index.
 - **L_r, L_c:** Lower bounds of row and column (usually 0).
-

Q5. Explain all the operations of Data Structures with examples.

Answer:

Operations on data structures are the fundamental ways we manipulate information. The six standard operations are:

1. **Traversing:** Visiting every element in the structure exactly once to process it.
 - *Example:* Printing all names in a phonebook list.
 2. **Insertion:** Adding a new data element to the structure.
 - *Example:* Adding a new student's record into a database.
 3. **Deletion:** Removing an existing data element from the structure.
 - *Example:* Deleting a cancelled appointment from a calendar.
 4. **Searching:** Finding the location of an element based on a specific key value.
 - *Example:* Searching for a specific book title in a library database.
 5. **Sorting:** Arranging the elements in a specific logical order (ascending or descending).
 - *Example:* Sorting exam scores from highest to lowest.
 6. **Merging:** Combining elements from two different sorted structures into a single sorted structure.
 - *Example:* Merging two separate department employee lists into one company-wide list.
-